

Seatwork 7.1 Errors, exceptions, debugging, and troubleshooting in JavaScript	
Course Code: CPE025 B	Program: BSCPE
Course Title: Software Design	Date Performed: 02/26/2026
Section: CPE23S4	Date Submitted: 02/26/2026
Name: Bautista, Jhon Hendricks	Instructor: Engr. Neal Barton James Matira
3. Procedure	
6.2.13 SECTION PRACTICE Exercise 1: Run the following code: <pre> 1 let end = 2; 2 for(let i=1; i 3 console.log(i); 4 }</pre> It should output the numbers 1 and 2 on the console. Use the debugger to make the program output the numbers 1, 2, 3, 4, and 5 instead. Do not modify the program code. Use only breakpoints and the option to modify variables. Exercise 2: Use the debugger to understand why the final result logged is equal to zero when on each iteration of <code>for</code>, a loop value of the variable result increases. Use <code>Watch</code> to track the changes in the selected variables. <pre> 1 let counter = 0; 2 let maxValue = 10; 3 let result = 1; 4 debugger; 5 for (counter = 0; counter < maxValue; counter++) { 6 console.log(result); 7 result *= maxValue - counter - 1; 8 } 9 console.log("Final result: ", result); 10</pre>	

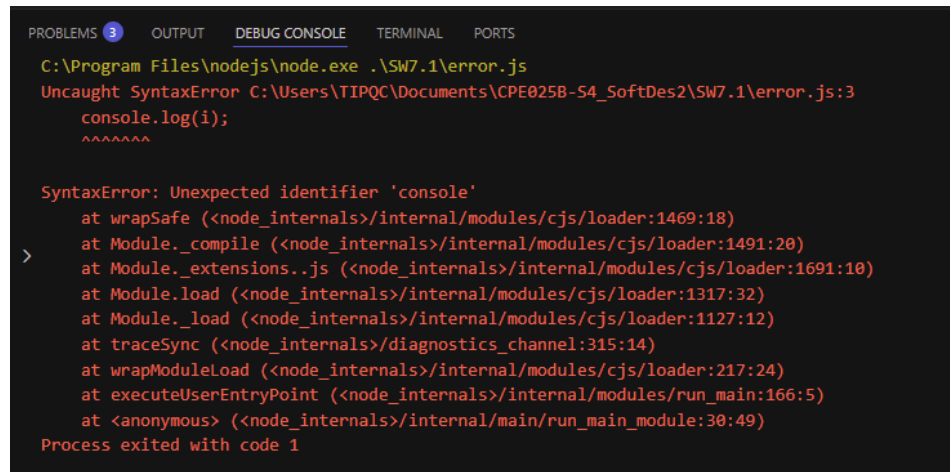
Exercise 3: Run the following code:

```
1 function max(array) {
2   let maxValue = array[1];
3   for(let i=1; i
4     if(array[i] > maxValue) {
5       maxValue = array[i];
6     }
7   }
8   return maxValue;
9 }
10 console.log( max([1, 4, 6, 2])); // -> 6
11 console.log( max([10, 4, 6, 2])); // -> 6
12
```

The function max should return the largest number from the array given as an argument. As you can see, in the second case it works incorrectly: instead of the value **10** we get **6**. Using the debugger, trace the max function execution step by step. Observe the values of the **i** and **maxValue** variables. Locate the problem and fix the code so that it works correctly.

4. Output

Exercise 1



The screenshot shows a Node.js error message in the console. The error is a 'SyntaxError: Unexpected identifier 'console'' occurring in a file named 'error.js' at line 3. The code snippet shown is 'console.log(i);'. The error message is preceded by 'Uncaught SyntaxError: C:\Users\TIPQC\Documents\CPE025B-S4_SoftDes2\SW7.1\error.js:3'. The stack trace shows the error originated from the 'wrapSafe' function in the Node.js internal modules, specifically in the 'cjs/loader' module. The process exited with code 1.

Figure 1. Initial Program Evaluation

The main problem shown in figure 1 is that it lacks the proper structure for a for loop and it has incorrect use of parenthesis and curly braces.

```
let end = 2;

for (let i = 1; i <= end; i++) {
  console.log(i);
}
```

Figure 2. Modification

Since we aim to create the loop and output 1 and 2 I fixed the programs syntax and followed the correct for loop structure

```
1 6.2.13 SECTION PRACTICE.js:4
2 6.2.13 SECTION PRACTICE.js:4
Live reload enabled. output.html:39
>
```

Figure 3. Output

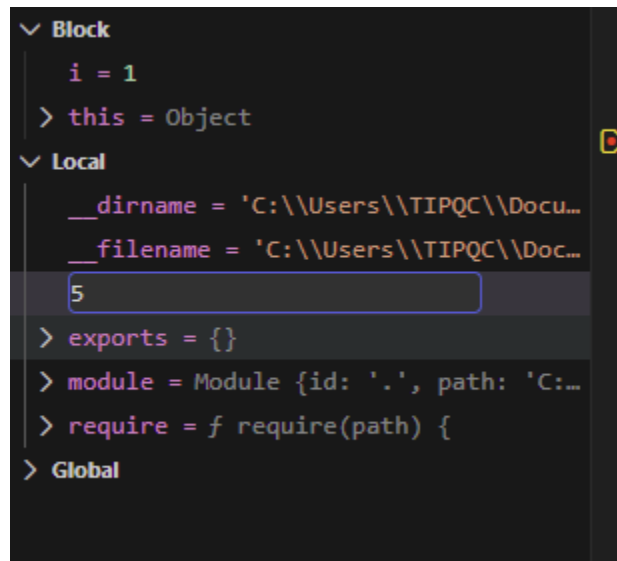


Figure 4. Using Debugger

Now using the debugger I can show the modification of the code even if I haven't changed the actual value of the variable. This will make the output from 1 to 5.

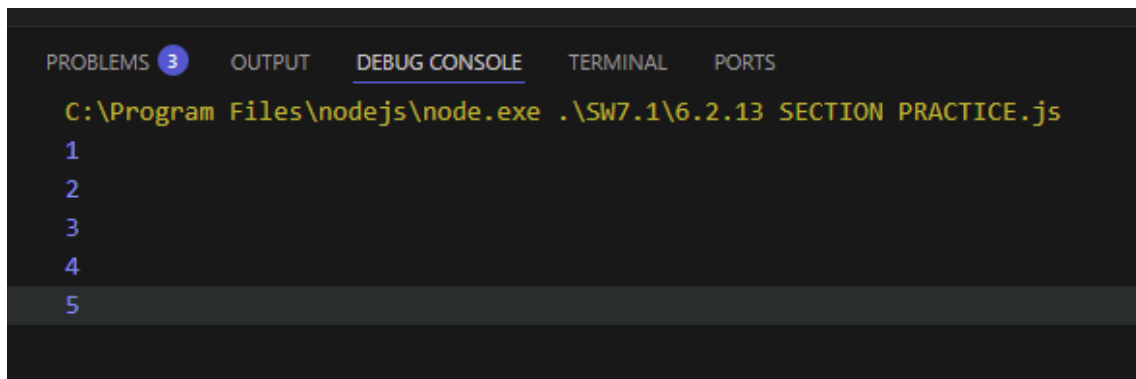


Figure 5. Output of Debugger

Exercise 2

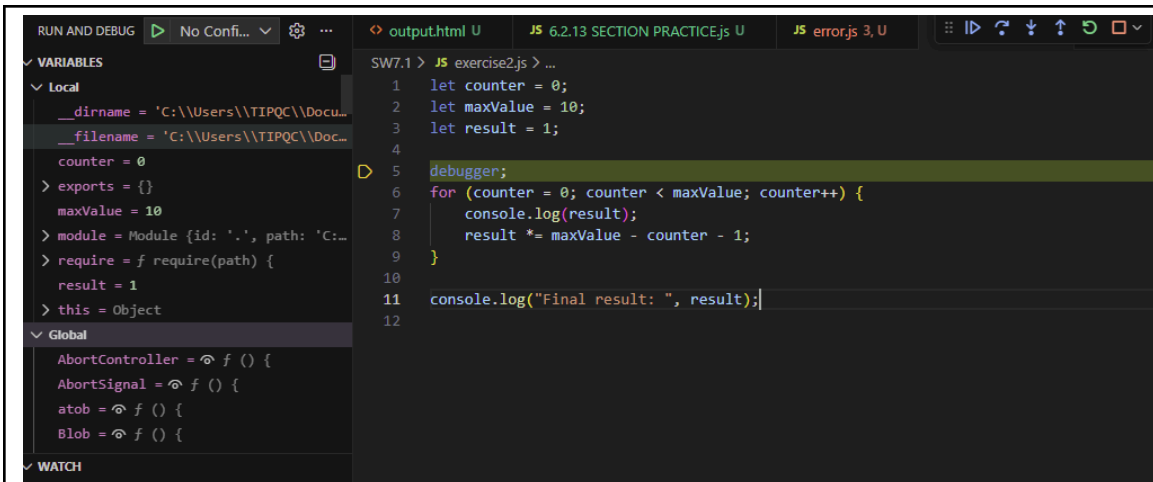


Figure 6. Debugging Problem 2

In using the debugger for problem 2 I observe that since the formula for increasing the value of result is the maxvalue - counter - 1, it will come to a point wherein the maxvalue and counter - 1 will be equal therefore becoming 0. This would then result in a result multiplied by 0 which is 0.

Exercise 3

```
function max(array) {
  let maxValue = array[1];
  for (let i = 1; i < array.length; i++) {
    if (array[i] > maxValue) {
      maxValue = array[i];
    }
  }
  return maxValue;
}

console.log( max([1, 4, 6, 2])); // -> 6
console.log( max([10, 4, 6, 2])); // -> 6
```

Figure 7. Fixed Program

Since the original program has syntax errors in its loop I fixed the structure of the for loop and applied the proper curly braces.

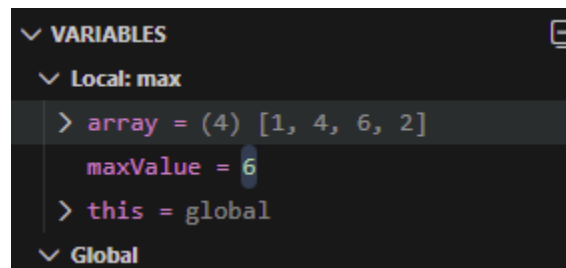


Figure 8. Debugger

Based on the structure of the indexing of the loop it will skip the value of 10 since it is placed at index 0 and the loop starts at index 1. This will make it so that the loop will fall at index of 6 in both of the arrays.

Answer the following Questions:

1. Discuss the stability and readability risks inherent in a program that relies solely on conditional logic (e.g., *if-else* checks) to manage errors, rather than utilizing a formal exception handling mechanism.
 - The usage of conditional logic when managing errors can harm the entire codebase since it adds more unnecessary code for the sake of handling the error. This can lessen the readability of the code since we will add more code and maybe even have nested operations which are hard to follow and debug. This is where the javascript debugger can help since we can explore the error and handle it without adding more code.
2. Analyze the specific operational risks, such as memory leaks or locked files, that arise when a developer fails to implement a *finally* block during interactions with external systems or databases.
 - If you fail to use the finally block the system can remain open which can cause memory leaks or locked files. This is because the data or process is still open even if it is closed. The unclosed resources can cause memory errors and can lock resources causing a locked file.

CONCLUSION

After performing this activity I was able to learn about debugging processes in javascript. For this activity I was able to utilize the debugger in javascript. In this activity I observe that javascript allows an isolated environment for error handling which can help in safely handling the errors in the code. The debugger allows us to view the different changes on the code and a variable's values and monitor it each step of the program. This allows for a better process for error handling.